MLOps-Based Anomaly Detection for Cloud Infrastructure Monitoring

Synopsis for Mini Project (ICS 554)

Bachelor of Technology
in
Computer Science and Engineering

Submitted by
Ujjawal Singh Bhalla (2400520100166)
Utkarsh Pandey (2400520100167)

Under the guidance of
Mr. Akash
Mrs. Renu Yadav



Department of Computer Science and Engineering

Institute of Engineering and Technology, Lucknow
Dr. APJ Abdul Kalam Technical University, Uttar Pradesh

2026–27

OpsLens — ML-Powered DevOps Monitoring & Anomaly Detection Platform

1. Introduction

Cloud-based applications rely on the continuous operation of distributed services, databases and supporting infrastructure. These systems generate large volumes of time-series metrics, including request rates, response latency, service availability and application error or crash rates. Unexpected changes in these measurements may indicate performance degradation, resource-related problems or service failures. Conventional monitoring systems commonly depend on fixed thresholds and manual inspection; however, such approaches may produce false alerts when normal system behaviour varies over time. An automated anomaly-detection mechanism can help identify unusual metric patterns and support timely investigation.

This project, titled "OpsLens — MLOps-Based Anomaly Detection for Cloud Infrastructure Monitoring", aims to develop a prototype that applies machine learning to infrastructure-related time-series data and organizes the model lifecycle through a repeatable MLOps workflow. The work uses the Microsoft Cloud Monitoring Dataset, which contains multiple metric-specific CSV time series. An Isolation Forest-based approach is used as the initial unsupervised learning method because it can identify unusual observations without requiring labelled examples for model training. The project focuses not only on generating anomaly predictions, but also on consistent preprocessing, model evaluation, artifact management and a deployable application architecture.

2. Related Works

Liu, Ting and Zhou introduced Isolation Forest, an unsupervised anomaly-detection algorithm that identifies unusual observations by isolating them through randomly generated partitioning trees [1]. Its relatively low computational cost makes it a suitable baseline for time-series anomaly detection when labelled examples are limited. The authors later presented isolation-based anomaly detection in greater detail, discussing its efficiency and applicability to high-dimensional data [2].

Research on the AIOps Challenge has made operational KPI time-series datasets available for studying anomaly detection in internet-service environments. Such datasets include labelled anomalous observations or intervals that can be used to evaluate detection methods [3]. Ren et al. described a time-series anomaly-detection service at Microsoft and highlighted the importance of integrating data processing, model development and operational execution into a systematic pipeline [4].

Zhang et al. investigated cross-dataset time-series anomaly detection for cloud systems, addressing the challenge of detecting abnormal behaviour when labelled data is limited across environments [5]. These studies provide the research foundation for OpsLens. Building on the use of unsupervised learning for operational metrics, the proposed project emphasizes a practical and reproducible workflow that connects model development with application-level monitoring. The implementation is intended as an academic prototype and is scoped to demonstrate the core concepts rather than reproduce a production-scale AIOps platform.

3. Existing / Proposed Methodology

Cloud infrastructure monitoring is commonly performed using threshold-based alerts, statistical techniques and machine-learning approaches. Static thresholds are easy to configure but may not adapt well to changing workloads, while more complex learning methods can require substantial data, computation and tuning. OpsLens proposes an Isolation Forest-based baseline to detect unusual behaviour in selected infrastructure metrics. The system is designed as a modular pipeline in which data preparation, model training, inference and visualization can be developed and tested independently.

The proposed system will follow the stages described below:



Initially, the Microsoft Cloud Monitoring Dataset is inspected to identify usable metric-specific time series and their timestamp, numeric value and optional label fields. The CSV files are processed through a common preprocessing routine that parses timestamps, converts metric values to numeric form, removes invalid records and arranges observations chronologically. Time-dependent features, such as rolling statistics, deviation from recent behaviour, point-to-point change and time-of-day encodings, are generated to represent the behaviour of each series. The data is divided chronologically into training and testing portions so that future observations are not used to train the model.

An individual StandardScaler and Isolation Forest pipeline is trained for each compatible time series. The trained models, preprocessing information and recent historical context are saved as reusable artifacts and indexed through a model manifest. During inference, a selected model processes incoming metric observations and returns anomaly predictions with anomaly scores; higher scores indicate more unusual observations and are not interpreted as calibrated probabilities. Where ground-truth labels are available, the predictions are evaluated using suitable classification measures, while time-series plots and score distributions are used for further inspection. The planned application will connect the Python inference component to a backend service and a web-based dashboard. Docker-based packaging and an automated build-and-test workflow will be used to make execution more consistent and reproducible.

4. Expected Results

The expected outcome is OpsLens, a prototype for identifying unusual behaviour in cloud infrastructure metrics through a repeatable machine-learning workflow. Preliminary implementation has established the dataset-processing and model-training pipeline, and an initial run has generated 60 per-series model artifacts. The project will extend this foundation into an integrated monitoring application. The expected deliverables include:

- A reusable data-processing and feature-engineering pipeline for metric-specific time-series CSV files.
- Trained Isolation Forest models, model metadata and a manifest for selecting the appropriate model during inference.
- An inference component that produces anomaly labels and scores, supported by evaluation summaries and visualizations of metric behaviour.
- A web-based monitoring dashboard and backend integration for submitting metric data and presenting detected anomalies.
- A containerized and reproducible application workflow, supported by testing and an evaluation report describing performance, limitations and possible improvements.

The quality of anomaly detection may vary across metric types because their normal behaviour, scale and anomaly patterns differ. Therefore, model performance will be reported using the available evaluation data rather than assuming a fixed accuracy in advance. The completed prototype is expected to demonstrate how machine learning can be incorporated into an MLOps-oriented monitoring workflow and provide a foundation for future work, including additional detection methods, automated retraining and deployment to a cloud environment.

5. Plan of Work

Objectives	Planned activity
1	Study infrastructure anomaly detection, review related research and inspect the Microsoft Cloud Monitoring Dataset.
2	Discover and preprocess metric-specific CSV time series; handle timestamps, invalid values and optional labels.
3	Develop time-series features and train per-series StandardScaler–Isolation Forest models. Initial run: 60 model artifacts.
4	Prepare model metadata, manifest, chronological test predictions and offline JSON-based inference.
5	Evaluate selected metric series and refine the inference workflow using normal and synthetic anomaly scenarios.
6	Integrate the Python inference component with the application backend and persist metric observations and anomaly events.
7	Develop the web dashboard for metric visualization, anomaly scores and detected events.
8	Package and test the complete application using Docker; establish CI/CD and prepare final documentation and demonstration.

References

[1] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in Proc. 8th IEEE Int. Conf. on Data Mining, 2008, pp. 413–422, doi: 10.1109/ICDM.2008.17.

[2] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation-Based Anomaly Detection," ACM Trans. Knowl. Discov. Data, vol. 6, no. 1, Art. no. 3, 2012, doi: 10.1145/2133360.2133363.

[3] NetMan AIOps, "KPI Anomaly Detection: 2018 AIOps Challenge Dataset," GitHub repository. [Online]. Available: https://github.com/NetManAIOps/KPI-Anomaly-Detection. [Accessed: Oct. 30, 2026].

[4] H. Ren et al., "Time-Series Anomaly Detection Service at Microsoft," in Proc. 25th ACM SIGKDD Int. Conf. on Knowledge Discovery & Data Mining, 2019, pp. 3009–3017, doi: 10.1145/3292500.3330680.

[5] X. Zhang et al., "Cross-dataset time series anomaly detection for cloud systems," in Proc. USENIX Annual Technical Conference (USENIX ATC), 2019.

Appendix A: Dataset Description and Structure

A.1 Dataset Overview

OpsLens uses the Microsoft Cloud Monitoring Dataset, which contains time-series data representing different application and cloud infrastructure metrics. The dataset is used to train and evaluate machine-learning models for detecting unusual patterns in system behaviour.

A.2 Dataset Structure

The dataset is organized into metric-specific directories, each containing multiple CSV files representing individual time series. The major metric categories include:

- Application Crash Rate
- Consumer Purchase Rate
- Data Ingress Rate

- E-commerce API Incoming Requests
- Middle-tier API Dependency Latency
- MongoDB Application and Machine Requests
- Service Unavailability

Each CSV file primarily contains timestamp and numerical metric-value observations, along with an optional anomaly label where available.

A.3 Data Processing and Utilization

The dataset is preprocessed by parsing timestamps, removing invalid records, sorting observations chronologically and handling duplicate entries. Additional features, including rolling mean, rolling standard deviation, deviation from recent averages and changes between consecutive observations, are extracted from the metric values.

The processed data is divided chronologically into training and testing sets. The project uses StandardScaler and Isolation Forest to train an individual anomaly-detection model for each compatible time series. Available anomaly labels are reserved for evaluation and are not used during unsupervised model training.

During the initial training run, the pipeline successfully generated 60 individual model artifacts, which are stored with their metadata and referenced through a model manifest for subsequent predictions.


```
{
  "model_id": "set-data-middle-tier-api-dependency-latency-outbound-01-f88e320f",
  "observations": [
    {
      "timestamp": "2018-07-07T23:00:00Z",
      "value": 56.033189
    },
    {
      "timestamp": "2018-07-08T00:00:00Z",
      "value": 56.502036
    },
    {
      "timestamp": "2018-07-08T01:00:00Z",
      "value": 57.251425
    },
    {
```


Image 1: Sample Request


```
{
  "success": true,
  "model_id": "set-data-middle-tier-api-dependency-latency-outbound-01-f88e320f",
  "source_file": "cloud-monitoring-dataset/data/middle-tier-api-dependency-latency/outbound-01.csv",
  "count": 48,
  "anomaly_count": 9,
  "score_semantics": "Higher anomaly_score means more anomalous; it is not a probability.",
  "predictions": [
    {
      "timestamp": "2018-07-07T23:00:00+00:00",
      "value": 56.033189,
      "anomaly": false,
      "anomaly_score": -0.16377510738581608,
      "decision_function": 0.16377510738581608
    },
    {
      "timestamp": "2018-07-08T00:00:00+00:00",
      "value": 56.502036,
      "anomaly": false,
      "anomaly_score": -0.15640236464523066,
      "decision_function": 0.15640236464523066
    },
```


Image 2: Sample Output (Anomaly Detection and Anomaly Score)